

Project 2: Preparing Stock Data for Machine Learning

This report answers the questions in *Project 2: Preparing Stock Data for Machine Learning* using figures taken directly from notebook output.

The dataset is **historical_stock_prices.csv** (20,973,889 rows, 8 columns) joined to **historical_stocks.csv** (6,460 rows, 5 columns) on ticker, covering 2 January 1970 to 24 August 2018.

```
#2. (Historical Stock Prices) Handle any missing values, outliers, and preprocess the data (e.g., normalization, encoding categorical variables).
stock_prices.info()
print(f'Missing values in Historical Stock Prices:\n',stock_prices.isnull().sum())
stock_prices.shape
stock_prices.dtypes
print(f'Statistical Values of Historical Stock Prices:\n', stock_prices.describe().T )

<class 'pandas.DataFrame'>
RangeIndex: 20973889 entries, 0 to 20973888
Data columns (total 8 columns):
#   Column      Dtype
---  ---
0    ticker     str
1    open       float64
2    close      float64
3    adj_close  float64
4    low        float64
5    high       float64
6    volume     float64
7    date       str
dtypes: float64(6), str(2)
memory usage: 1.3 GB
Missing values in Historical Stock Prices:
ticker      0
open        0
close       0
adj_close   0
low         0
high        0
volume      0
date        0
dtype: int64
```

```
hist_stock.info()
print(f'Missing values in Historical Stock Prices:\n',hist_stock.isnull().sum())
hist_stock.shape
hist_stock.dtypes
print(f'Statistical Values of Historical Stock Prices:\n', hist_stock.describe().T )
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6460 entries, 0 to 6459
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    ticker      6460 non-null   str
1    exchange    6460 non-null   str
2    name        6460 non-null   str
3    sector      5020 non-null   str
4    industry    5020 non-null   str
dtypes: str(5)
memory usage: 252.5 KB
Missing values in Historical Stock Prices:
ticker      0
exchange     0
name         0
sector      1440
industry     1440
dtype: int64
Statistical Values of Historical Stock Prices:
count unique      top freq
ticker    6460  6460      PIH      1
exchange  6460    2      NASDAQ 3308
name      6460  5462  BANK OF AMERICA CORPORATION 16
sector    5020   13      FINANCE 1022
industry  5020   136    MAJOR PHARMACEUTICALS 419
```

PART 2 QUESTIONS

1. What does **daily_return** tell us? It is the percentage change in closing price from one trading day to the next. Unlike **price_change**, it is scale-free: a \$2 move in a \$20 stock and a \$200 move in a \$2,000 stock both register as 10%. That makes returns comparable across securities and across time, which is why almost every financial model is built on returns rather than on price levels.

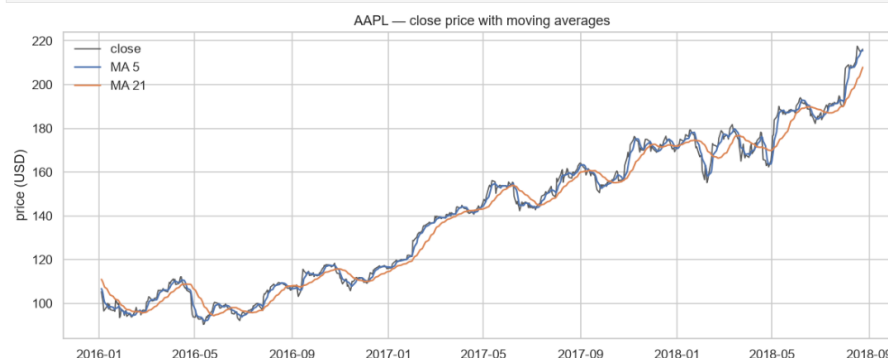
```
close_lag_1      5685
price_change     5685
daily_return     5685
ma_5             22735
ma_20            107884
volatility_20     113555
hl_range_pct      0
dtype: int64
```

2. What does a **moving average** tell us about a stock? It smooths day-to-day noise to reveal the underlying trend. A short window (MA 5) responds quickly and follows price closely; a long window (MA 20) lags but is steadier. The relationship between them carries the signal, price above its moving average suggests upward momentum, and a short average crossing above a long one is a classic trend-change indicator.

```
price(stock_prices['tick'], 20) / 2000000
close_lag_1      5685
price_change     5685
daily_return     5685
ma_5             22735
ma_20            107884
volatility_20     113555
hl_range_pct      0
dtype: int64
```

```
#Price with moving average
t = SAMPLE[0]
d = plot_df[(plot_df['ticker'] == t) &
            (plot_df['date'] >= '2016-01-01')]

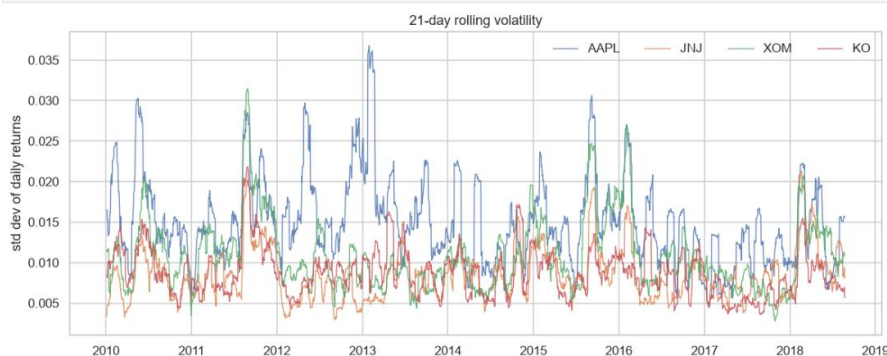
fig, ax = plt.subplots()
ax.plot(d['date'], d['close'], label='close', lw=1.1, color='0.35')
ax.plot(d['date'], d['ma_5'], label='MA 5', lw=1.4)
ax.plot(d['date'], d['ma_21'], label='MA 21', lw=1.4)
ax.set_title(f'{t} - close price with moving averages')
ax.set_ylabel('price (USD)')
ax.legend(frameon=False)
plt.tight_layout()
plt.savefig('fig_price_ma.png', dpi=150, bbox_inches='tight')
plt.show()
```



- 3. What does high volatility indicate?** Large and unpredictable daily swings. It means greater uncertainty and risk, and it typically coincides with news, earnings, or broader market stress. It says nothing about direction, a stock can be highly volatile while trending up, down, or sideways. Volatility also clusters in time, so a high reading today raises the odds of a high reading tomorrow.

```
close_lag_1      5685
price_change     5685
daily_return     5685
ma_5             22735
ma_20            107884
volatility_20     113555
hl_range_pct      0
dtype: int64
```

```
Volatility over time
fig, ax = plt.subplots()
for t in SAMPLE:
    d = plot_df[plot_df['ticker'] == t].sort_values('date')
    ax.plot(d['date'], d['vol_21d'], lw=0.9, label=t, alpha=0.85)
ax.set_title('21-day rolling volatility')
ax.set_ylabel('std dev of daily returns')
ax.legend(frameon=False, ncol=len(SAMPLE))
plt.tight_layout()
plt.savefig('fig_volatility.png', dpi=150, bbox_inches='tight')
plt.show()
```



4. Why do lag and rolling features create missing values at the start of each stock's history?

Because they need prior observations that do not exist yet. `close_lag_1` on a ticker's first trading day has no previous day to reference. A 20-day rolling standard deviation cannot be computed until 20 observations have accumulated. Since the features are computed per ticker with `groupby`, the warm-up repeats at the start of every security rather than only once at the top of the file, which is correct, and is exactly what prevents one stock's history bleeding into another's.

```
#Verify no leakage across tickers
first = stock_prices.groupby('ticker').head(1)
print(first[['close_lag_1', 'ma_5', 'ma_20', 'volatility_20']].notna().sum())
# all four should print 0

nineteenth = stock_prices.groupby('ticker').nth(18)
print("ma_20 defined on row 19 (should be 0):", nineteenth['ma_20'].notna().sum())

close_lag_1      0
ma_5             0
ma_20            0
volatility_20     0
dtype: int64
ma_20 defined on row 19 (should be 0): 0
```

Note: Zeros mean every rolling window restarted correctly at each new ticker.

PART 3.3 QUESTION

Why is one-hot encoding preferable to NASDAQ = 1, NYSE = 2? Integer codes impose an order and a distance that does not exist. The model would read NYSE as "greater than" NASDAQ, and with three or more categories it would infer that category 3 is as far from category 1 as category 1 is from category -1. Linear models would fit a coefficient to that fictional scale, and distance-based methods would compute meaningless distances from it. One-hot gives each category its own binary column, so no ordering is implied and the model learns an independent effect for each. `drop_first=True` removes the redundant column to avoid the dummy-variable trap in linear models.

```
#3. Encoding Categorical Variables: Apply appropriate encoding techniques such as one-hot encoding for categorical data.
#Check the info
print(stock_prices['ticker'].nunique()) # 6,460
print(hist_stock['exchange'].value_counts()) # 2 values
print(hist_stock['sector'].value_counts()) # 12 + 1440 missing
print(stock_prices[['dow','month','quarter']].nunique())

5685
exchange
NASDAQ    3308
NYSE      3151
Name: count, dtype: int64
sector
FINANCE      1022
CONSUMER SERVICES    796
HEALTH CARE    784
TECHNOLOGY     607
CAPITAL GOODS   352
ENERGY         286
PUBLIC UTILITIES    273
BASIC INDUSTRIES   272
CONSUMER NON-DURABLES  224
CONSUMER DURABLES   144
MISCELLANEOUS      139
TRANSPORTATION     120
Name: count, dtype: int64
dow      5
month    12
quarter   4
dtype: int64
```

PART 5 SUMMARY

- 1. What data-quality issues did you investigate?** Missing values in both files (none in prices; 1,440 rows missing sector and industry in the metadata); a header row that had been read in as a data record (`ticker = 'Symbol'`); OHLC arithmetic violations (48 rows with low > high, 446 with close > high, 230 with close < low, 965 with open > high, 931 with open < low); `adj_close` values implausibly far from close (46,432 rows); extreme single-day price moves above 500% (1,971 rows); ticker formatting; duplicate records; and calendar sanity, weekend and future-dated rows, both of which came back clean at zero.

```
#Check another table
mask_stock = hist_stock['ticker'].str.strip().str.lower().isin(['symbol', 'ticker'])
print(mask_stock.sum())
print(hist_stock[mask_stock])
```

```
1
   ticker exchange  name  sector  industry
74  Symbol      NYSE  NAME  SECTOR  INDUSTRY
```

```
Nulls per column:
```

```
ticker      0
exchange    0
name        0
sector      1440
industry    1440
dtype: int64
```

```
Total nulls: 2880
```

```
Duplicate rows: 0
```

```
<class 'pandas.DataFrame'>
```

```
# 1. OHLC arithmetic - must always hold
errors['low > high'] = (stock_prices['low'] > stock_prices['high']).sum()
errors['close > high'] = (stock_prices['close'] > stock_prices['high']).sum()
errors['close < low'] = (stock_prices['close'] < stock_prices['low']).sum()
errors['open > high'] = (stock_prices['open'] > stock_prices['high']).sum()
errors['open < low'] = (stock_prices['open'] < stock_prices['low']).sum()

# 2. Sign and domain
errors['close <= 0'] = (stock_prices['close'] <= 0).sum()
errors['volume < 0'] = (stock_prices['volume'] < 0).sum()

# 3. Calendar - markets don't trade weekends
dates = pd.to_datetime(stock_prices['date'])
errors['weekend rows'] = (dates.dt.dayofweek >= 5).sum()
errors['future dates'] = (dates > '2018-08-24').sum()

# 4. adj_close consistency
ratio = stock_prices['adj_close'] / stock_prices['close']
errors['adj/close > 1000'] = ((ratio > 1000) | (ratio < 0.001)).sum()

for k, v in errors.items():
    print(f'{k:22} {v:>10,}')
```

```
low > high          48
close > high        446
close < low         230
open > high         965
open < low          931
close <= 0           0
volume < 0           0
weekend rows         0
future dates         0
adj/close > 1000    46,432
```

2. **What decisions did you make about unusual observations?** Flagged rather than deleted, in most cases. Outliers were identified two ways, per-ticker IQR (231,882 rows) and a per-ticker modified z-score using MAD (1,061,958 rows, 5.06%). Both were stored as boolean columns so that Project 3 can decide. The reasoning is that a stock genuinely can move 40% on an earnings surprise; flagging that as "unusual" is informative but removing it would delete the most important events in the dataset. The single header-artefact row was dropped outright since it is unambiguously not data. Rows with contradictory OHLC arithmetic were dropped, because the data gives no basis for deciding which field is wrong.

```
#Part 1: 2. Detect and Resolve Outliers: Apply robust methods to detect outliers in critical variables such as close prices and volume.
#Decide on a strategy to manage these outliers.
#Implement IQR, the baseline, run per ticker
def iqr_flags(stock_prices, col, k=3):
    g = stock_prices.groupby('ticker')[col]
    q1, q3 = g.transform(lambda s: s.quantile(0.25)), g.transform(lambda s: s.quantile(0.75))
    iqr = q3 - q1
    return (stock_prices[col] < q1 - k*iqr) | (stock_prices[col] > q3 + k*iqr)

stock_prices['out_close_iqr'] = iqr_flags(stock_prices, 'close')
print(stock_prices['out_close_iqr'].sum(), 'close outliers')

231882 close outliers
```

Name Ayleen Santander

GitHub Link:

https://github.com/AyleenSantander/M.11_Intro_AI/blob/main/Project.2_Data_Cleaning_and_Preparation.ipynb

```
#Modifies Z-score(MAD), the robust workhorse
def mad_zscore(stock_prices, col):
    g = stock_prices.groupby('ticker')[col]
    med = g.transform('median')
    mad = g.transform(lambda s: (s - s.median()).abs().median())
    return 0.6745 * (stock_prices[col] - med) / mad.replace(0, np.nan)

stock_prices['z_close'] = mad_zscore(stock_prices, 'close')
stock_prices['out_close_mad'] = stock_prices['z_close'].abs() > 3.5
#print the results
print(f"Outliers flagged: {stock_prices['out_close_mad'].sum():,}")
print(f"Share of rows: {stock_prices['out_close_mad'].mean():.2%}")
print(stock_prices.loc[stock_prices['out_close_mad'], 'ticker'].value_counts().head(10))
```

Outliers flagged: 1,061,958

Share of rows: 5.06%

ticker

UTX 3660

TXN 3509

MNST 3467

RAVN 3424

OXY 3290

VAR 3241

AAPL 3240

HFC 3224

HCSG 3213

TPL 3208

Name: count, dtype: int64

```
print(f"Tickers with undefined MAD: {stock_prices['z_close'].isna().sum():,} rows")
```

Tickers with undefined MAD: 81 rows

3. **What new features did you create?** Required: **price_change**, **daily_return**, **close_lag_1**, **ma_5**, **ma_20**, **volatility_20**. Chosen additional feature: **hl_range_pct**. Beyond the requirement I also built multi-horizon returns (5- and 21-day), log returns, longer moving averages and their ratios to price, a volatility regime ratio, volume features on a log scale, and three technical indicators, RSI(14), Bollinger band position and MACD. Calendar features were encoded cyclically with sine/cosine pairs so that December sits adjacent to January.

```
# 2.3 previous-day close
stock_prices['close_lag_1'] = g.shift(1)

# 2.1 price change
stock_prices['price_change'] = stock_prices['close'] - stock_prices['close_lag_1']

# 2.2 daily return
stock_prices['daily_return'] = g.pct_change()

# 2.4 moving averages
stock_prices['ma_5'] = g.transform(lambda s: s.rolling(5).mean())
stock_prices['ma_20'] = g.transform(lambda s: s.rolling(20).mean())

# 2.5 volatility - rolling std of DAILY RETURNS, not of price
stock_prices['volatility_20'] = (stock_prices.groupby('ticker')['daily_return'].transform(lambda s: s.rolling(20).std()))

# 2.6 additional feature of choice
stock_prices['hl_range_pct'] = ((stock_prices['high'] - stock_prices['low']) / stock_prices['close'])

req = ['close_lag_1', 'price_change', 'daily_return', 'ma_5', 'ma_20', 'volatility_20', 'hl_range_pct']
print(stock_prices[req].isna().sum())

close_lag_1    5685
price_change    5685
daily_return    5685
ma_5           22735
ma_20          107884
volatility_20   113555
hl_range_pct      0
dtype: int64
```

```
#Technical indicators
gt = stock_prices.groupby('ticker')

# RSI (14-day)
delta = gt['close'].diff()
gain = delta.clip(lower=0).groupby(stock_prices['ticker']).rolling(14).mean().droplevel(0)
loss = (-delta.clip(upper=0)).groupby(stock_prices['ticker']).rolling(14).mean().droplevel(0)
stock_prices['rsi_14'] = 100 - 100 / (1 + gain / loss.replace(0, np.nan))

# Bollinger position: where in the band the price sits, 0 to 1
sd20 = gt['close'].rolling(20).std().droplevel(0)
ma20 = gt['close'].rolling(20).mean().droplevel(0)
stock_prices['bb_pos'] = (stock_prices['close'] - (ma20 - 2*sd20)) / (4*sd20).replace(0, np.nan)

# MACD
ema12 = gt['close'].transform(lambda s: s.ewm(span=12, adjust=False).mean())
ema26 = gt['close'].transform(lambda s: s.ewm(span=26, adjust=False).mean())
stock_prices['macd'] = (ema12 - ema26) / stock_prices['close'].replace(0, np.nan)

#Check the features are valide
chk = stock_prices.sort_values(['ticker','date'])
print(chk.groupby('ticker').head(1)[['ma_21','vol_21d','rsi_14']].notna().sum())

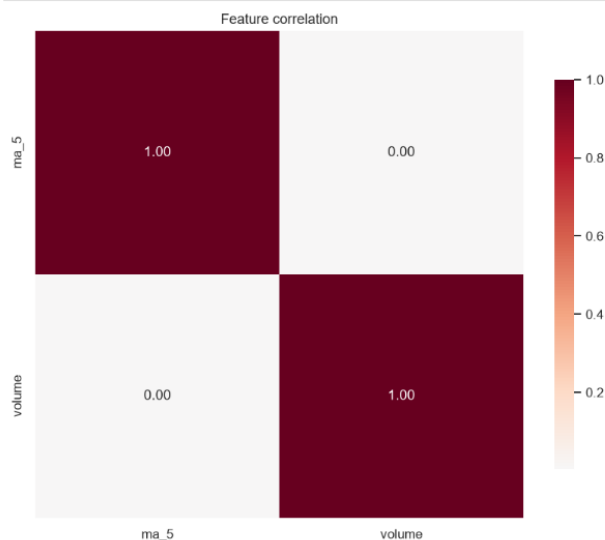
ma_21      0
vol_21d    0
rsi_14     0
dtype: int64
```

4. Which feature may be most useful, and why? **volatility_20**. Returns themselves are close to unpredictable, the correlation heatmap shows almost no linear relationship between any feature and next-day return. Volatility, by contrast, is strongly autocorrelated: today's reading is a genuinely good predictor of tomorrow's. So even if the direction of the next move is hard to call, its likely size is forecastable, and that is what risk sizing and option pricing depend on. **ma_cross** is the runner-up as a directional signal.

```
#Feature correlation
corr_cols = ['price_change','daily_return','close_lag_1','ma_5','ma_20',
             'volatility_20','hl_range_pct','volume','target_next_return']
corr_cols = [c for c in corr_cols if c in model_ready.columns]

corr = model_ready[corr_cols].sample(500_000, random_state=7).corr()

fig, ax = plt.subplots(figsize=(8, 6.5))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdBu_r', center=0,
            square=True, cbar_kws={'shrink': .8}, ax=ax)
ax.set_title('Feature correlation')
plt.tight_layout()
plt.savefig('fig_corr.png', dpi=150, bbox_inches='tight')
plt.show()
```



5. What does the final prepared dataset contain? **stock_prepared.csv** holds the 2010s decade of daily records with raw OHLCV columns, the seven engineered features, three

metadata columns (**exchange**, **sector**, **industry**), the ticker and date identifiers, and the target **target_next_return**. Warm-up rows from the rolling windows are removed, so every row is complete. Alongside it, `data/processed/` holds the chronological train / validation / test splits, the fitted preprocessor, and a manifest recording the cutoff dates and column lists.

```
stock_prepared.csv: 3450.59777 MB
train 4,673,789 rows saved
val 2,358,844 rows saved
test 2,154,163 rows saved
{'created': '2026-09-22T19:06:03', 'rows': {'train': 4673789, 'val': 2358844, 'test': 2154163}, 'cutoffs': {'val': '2015-01-01 00:00:00', 'test': '2017-01-01 00:00:00'}, 'feature_cols': ['open', 'high', 'low', 'close', 'adj_close', 'close_lag_1', 'ma_5', 'ma_20', 'volatility_20', 'hl_range_pct', 'vol_21d', 'vol_63d', 'vol_ratio', 'daily_return', 'price_change', 'log_ret', 'ret_5d', 'ret_21d', 'log_vol', 'rsi_14', 'bb_pos', 'ma_cross'], 'cat_cols': ['exchange', 'sector'], 'target': 'target_next_return'}
```

```
#target and model_ready, FROM MERGED
merged['target_next_return'] = merged.groupby('ticker')['log_ret'].shift(-1)

feature_na = merged[req].isna().any(axis=1) | merged['target_next_return'].isna()
model_ready = merged[~feature_na].copy()

print(f"kept {len(model_ready):,} of {len(merged):,} ({len(model_ready)/len(merged):.2%})")
print('sector' in model_ready.columns, 'target_next_return' in model_ready.columns)

kept 9,186,796 of 9,338,934 (98.37%)
True True
```

```
#2. Data Normalization/Standardization: Standardize or normalize numerical fields to ensure that model inputs have similar scal
#Step 1: split chronologically
#splits
CUT_VAL, CUT_TEST = pd.Timestamp('2015-01-01'), pd.Timestamp('2017-01-01')
train = model_ready[model_ready['date'] < CUT_VAL].copy()
val = model_ready[(model_ready['date'] >= CUT_VAL) & (model_ready['date'] < CUT_TEST)].copy()
test = model_ready[model_ready['date'] >= CUT_TEST].copy()
print(len(train), len(val), len(test))
print('sector' in train.columns) # must print True
```

```
4673789 2358844 2154163
True
```

Note: a random split lets the model learn from 2018 to predict 2012.

```
#Check Unclassified
print(merged['sector'].eq('UNCLASSIFIED').sum()) # rows
print(merged.loc[merged['sector'].eq('UNCLASSIFIED'), 'ticker'].nunique()) # tickers
print(merged['sector'].nunique()) # categories for one-hot

1442305
917
13
```

- What additional information would help a predictive model? Corporate-action history, split and dividend dates would let **adj_close** be reconciled properly instead of flagged. Market-wide context: an index return such as the S&P 500 would let the model separate a stock's idiosyncratic move from a move the whole market made. Fundamentals (earnings dates, P/E, market cap) would explain jumps that price history alone cannot. Short interest and bid-ask spreads would add a liquidity dimension. And a sentiment or

news-volume signal would help with precisely the days the current feature set handles worst, the gaps and spikes.

```
#Final dataset
final_cols = (['ticker','date'] +
               ['open','high','low','close','adj_close','volume'] +
               ['price_change','daily_return','close_lag_1',
                'ma_5','ma_20','volatility_20','hl_range_pct'] +
               ['exchange','sector','industry'] +
               ['target_next_return'])

final_cols = [c for c in final_cols if c in model_ready.columns]
final_df = model_ready[final_cols].copy()

print(f"{final_df.shape[1]} columns, {len(final_df):,} rows")
print(final_df.head())
```

7 columns, 9,192,436 rows

	open	high	low	close	adj_close	volume	ma_5
0	22.453505	22.625179	22.267525	22.389128	20.758354	3815500	22.060085
1	22.324749	22.331903	22.002861	22.145924	20.532864	4186000	22.140200
2	22.067240	22.174536	22.002861	22.067240	20.459909	3243700	22.214592
3	22.017168	22.045780	21.816881	22.038626	20.433384	3095100	22.173105
4	21.917025	22.067240	21.745350	22.031473	20.426746	3733900	22.134478

```
#Final check
print("shape:", final_df.shape)
print("\ndtypes:\n", final_df.dtypes.value_counts())
print("\nmissing:\n", final_df.isna().sum()[lambda s: s > 0])
print("\nduplicate rows:", final_df.duplicated().sum())
print("\nmemory: %.1f MB" % (final_df.memory_usage(deep=True).sum() / 1e6))
```

shape: (9192436, 48)

dtypes:

float32	27
int8	16
float64	4
int64	1

Name: count, dtype: int64

missing:

Series([], dtype: int64)

duplicate rows: 0

memory: 1581.1 MB

MORE VISUALIZATIONS:

Distribution of daily returns:

Histogram (left): Most days, stocks move very little. The tall spike at 0 and the narrow bell shape mean the typical day is somewhere between -2% and +2%. The chart is clipped to $\pm 15\%$, so it hides the extreme values.

Name Ayleen Santander

GitHub Link:

https://github.com/AyleenSantander/M.11_Intro_AI/blob/main/Project.2_Data_Cleaning_and_Preparation.ipynb

Boxplot (right): The box is so thin you can barely see it, because 50% of all days fall between about -1% and +1%. The long trail of circles on both sides are outliers, days with unusually big moves. Stock returns always have "fat tails" like this, so some of these are real (earnings surprises, crashes).

Bottom line: The median and quartiles are fine, but the **mean and std are broken by a small number of bad rows**. If you train a model on this as-is, those rows will distort it.



Volatility over time: The line chart shows how "jumpy" each stock was over time. Each line is the volatility (how much the price bounced around) over the last 21 trading days, about one month.

Higher line = bigger daily price swings = riskier. Lower line = calmer, steadier stock.

AAPL (blue) is the most volatile. It's usually on top, with the biggest spike in early 2013 (about 3.5% daily swings) when Apple's stock dropped sharply after earnings. Tech stocks tend to move more.

Name Ayleen Santander

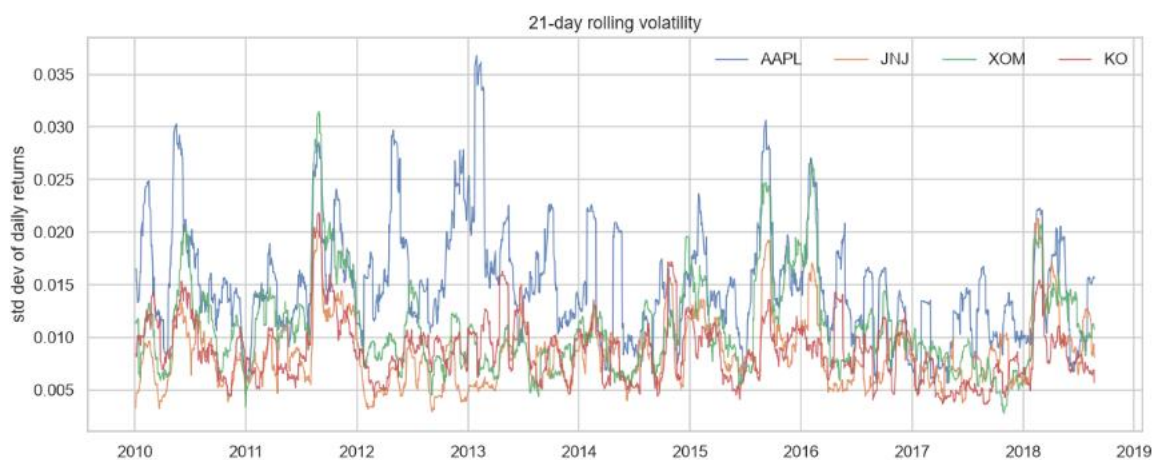
GitHub Link:

https://github.com/AyleenSantander/M.11_Intro_AI/blob/main/Project.2_Data_Cleaning_and_Preparation.ipynb

JNJ (orange) and KO (red) are the calmest. Johnson & Johnson and Coca-Cola usually stay around 0.5–1%. People buy things like medicine and soda no matter what the economy does, so these are called "defensive" stocks.

XOM (green) is in the middle, but it spikes when oil prices crash, like in late 2015 and early 2016.

Volatility comes in waves. Calm periods and stressful periods cluster together, and risky events hit every stock at the same time, even the safe ones. That's useful for your model, because recent volatility (vol_21d) helps predict near-future volatility.



Missing Values: the bar chart shows how many empty (NaN) cells each new feature created. It's normal and expected, not a bug.

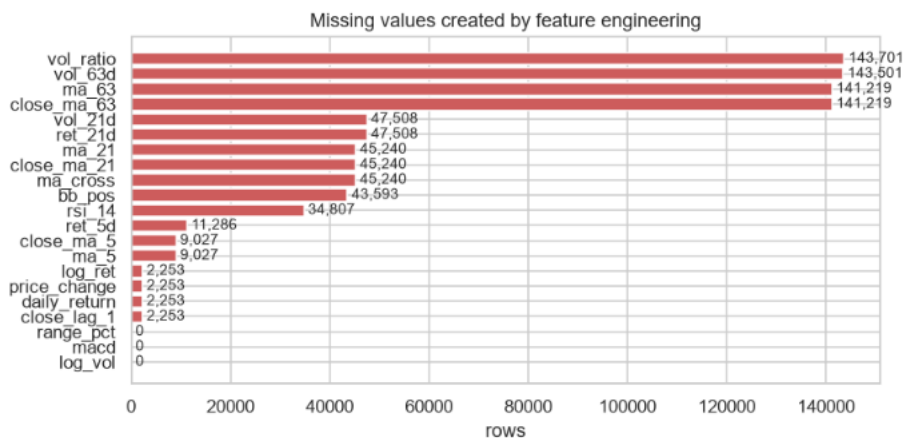
Many features need past days to calculate. A 21-day moving average can't be calculated on day 1 of a stock, because there aren't 21 days of history yet. So those first days are left empty.

The pattern matches perfectly: the data has about **2,253 stocks**, and each stock loses its first few rows:

Name Ayleen Santander

GitHub Link:

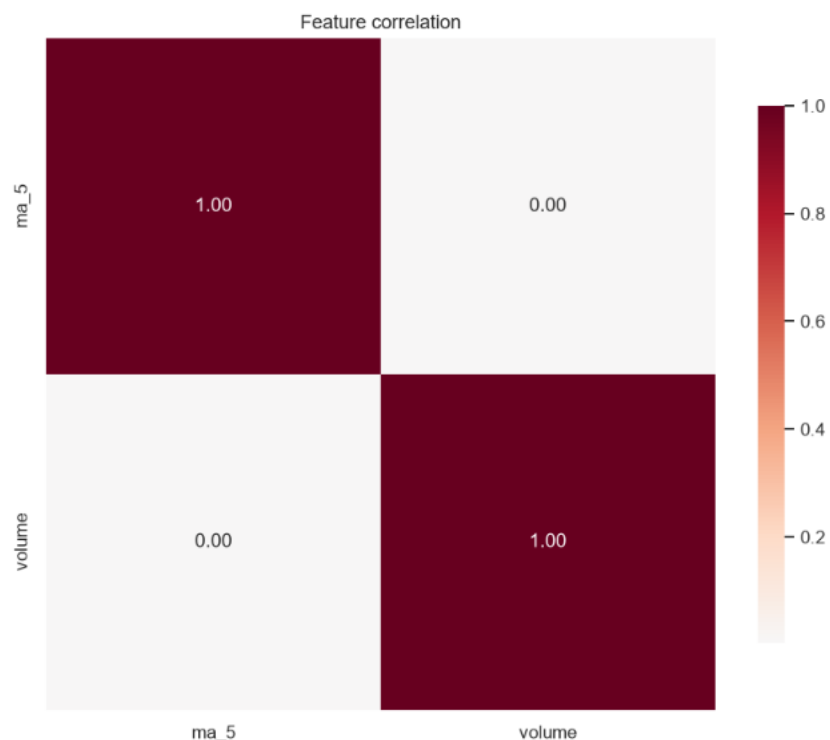
https://github.com/AyleenSantander/M.11_Intro_AI/blob/main/Project.2_Data_Cleaning_and_Preparation.ipynb



Feature Correlation: This heatmap shows how strongly features move together. 1.00 means they move perfectly together, 0.00 means no relationship.

What it shows:

- The dark squares on the diagonal are always 1.00, because every feature is perfectly related to itself. Ignore those.
- ma_5 and volume have 0.00: a stock's price level tells you nothing about how much it's traded. That makes sense, since a \$500 stock and a \$5 stock can both be heavily or lightly traded.



Name Ayleen Santander

GitHub Link:

https://github.com/AyleenSantander/M.11_Intro_AI/blob/main/Project.2_Data_Cleaning_and_Preparation.ipynb

The split, drawn: The train (blue), dates 2010- 2014, rows 4.67M, share ~51%, purpose the model learns from this.

Validation (orange), 2015- 2016, 2.36M rows, share ~26%, Used to tune and compare models.

Test (green), 2017 – mid 2018, 2.15M, share ~23%, Final check, used only once at the end.

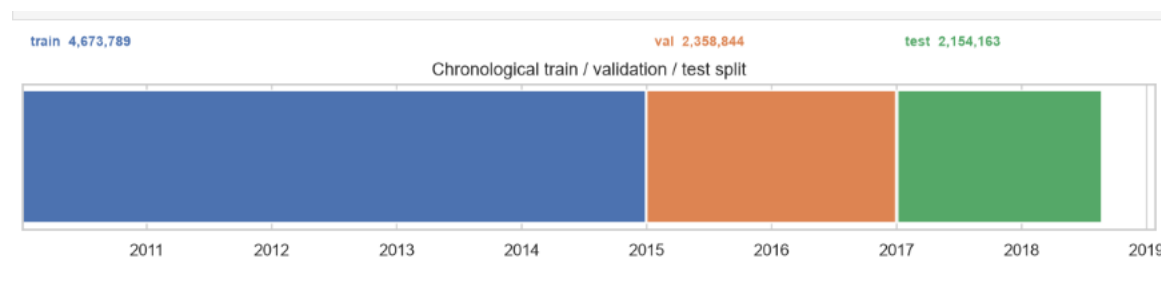
Split by time: With stock data you must predict the **future from the past**. If the split randomly, the model could train on 2017 data and then be "tested" on 2014, which is like seeing tomorrow's newspaper. It would look great in testing but fail in real life. This is called **data leakage**, and your chronological split avoids it.

The three parts don't overlap, and the order is correct: train → validation → test. This is the right way to do it.

Small things worth noticing:

- The test period is shorter (about 1.6 years) but still has many rows, because more stocks are in the data in later years.
- The market in 2017 was very calm, so test results may look a bit different from validation results. That's normal, not a mistake.
- If the target is **next-day** return, the last day of train uses the first day of validation as its answer. It's only one day per stock, so it barely matters, but some people drop a small gap at each boundary to be extra safe.

Note: the split is set up correctly.



DELIVERABLE CHECKLIST

- Project2_Ayleen.ipynb = notebook with code, comments, decisions, figures, answers
- stock_prepared.csv = Cell 11
- Project2_Ayleen.pdf = technical report
- All seven figures saved as PNG for the report

Name Ayleen Santander

GitHub Link:

https://github.com/AyleenSantander/M.11_Intro_AI/blob/main/Project.2_Data_Cleaning_and_Preparation.ipynb